

RedNorte

Servicio Público de Salud

Plataforma Inteligente para la Gestión de Listas de Espera Hospitalarias

Autor: Sebastián Godoy

Fecha: Julio 2026

Institución: Instituto Profesional DuocUC

Asignatura: DSY1106 — Desarrollo Fullstack III

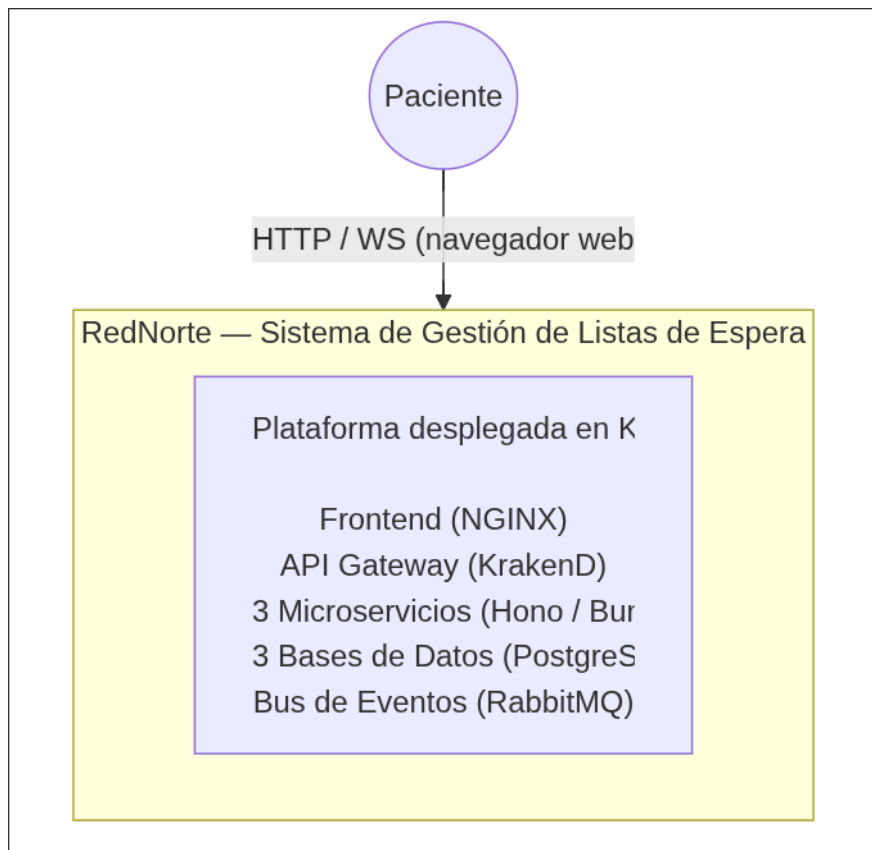
1. Diseño de la Arquitectura de Microservicios.....	4
1.1. Contexto del Problema.....	4
1.2. Arquitectura Propuesta.....	4
1.3. Autenticación y Seguridad.....	6
1.4. Escalabilidad y Sostenibilidad.....	6
2. Decisiones en el Desarrollo de Componentes Backend y Frontend.....	7
2.1. Decisiones de Backend.....	7
2.2. Decisiones de Frontend.....	8
2.3. Aciertos y Mejoras Identificadas.....	8
3. Aplicación de Patrones de Diseño.....	9
3.1. Repository Pattern.....	9
3.2. Service Layer Pattern.....	9
3.3. Centralized Error Handler.....	9
3.4. Observer / Pub-Sub Pattern.....	10
3.5. Atomic Store Pattern (Frontend).....	10
4. Estrategia de Branching y Gestión de Versiones.....	11
4.1. Estrategia Adoptada.....	11
4.2. Flujo de Trabajo.....	11
4.3. Convención de Commits.....	11
4.4. Impacto en el Trabajo en Equipo.....	12
5. Integración de Componentes Backend, Frontend y Base de Datos.....	13
5.1. Comunicación entre Servicios.....	13
5.2. Manejo de Datos.....	13
5.3. Desafíos de Integración.....	13
6. Pruebas Unitarias y Aseguramiento de la Calidad.....	14
6.1. Estrategia de Pruebas.....	14
6.2. Estrategia de Mocking.....	15
6.3. Impacto en la Calidad del Código.....	15
6.4. Dificultades y Estrategias de Optimización.....	15
7. Conclusión.....	17

1. Diseño de la Arquitectura de Microservicios

1.1. Contexto del Problema

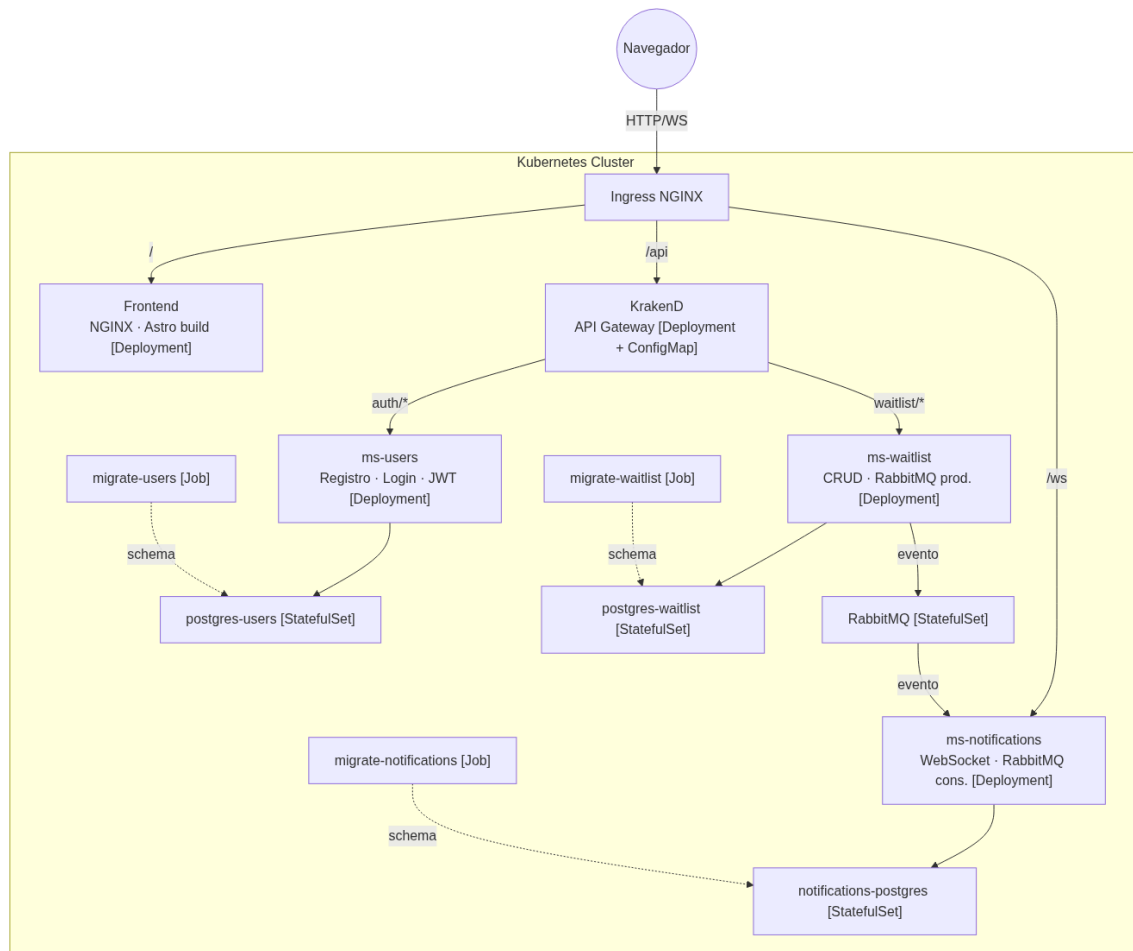
El Servicio Público de Salud RedNorte administra una red de hospitales, centros de atención primaria y clínicas especializadas en el norte del país. Uno de sus principales desafíos es la gestión eficiente de las listas de espera para consultas médicas, procedimientos y cirugías. Los sistemas existentes presentan problemas de transparencia: los pacientes no conocen el estado de su solicitud y el personal médico carece de herramientas para priorizar de manera eficiente.

La solución propuesta es una plataforma digital basada en microservicios que centraliza la gestión de listas de espera, optimiza la asignación de horas médicas y proporciona visibilidad a los pacientes sobre el estado de sus solicitudes mediante notificaciones en tiempo real.



1.2. Arquitectura Propuesta

La arquitectura se compone de cuatro capas: presentación, API Gateway, lógica de negocio y persistencia. Los componentes se ejecutan como contenedores Docker, orquestados con Docker Compose para desarrollo y Kubernetes para producción.



Frontend: Astro 4 con Preact 10 para componentes interactivos, Nanostores para estado global y Tailwind CSS 3. En desarrollo corre en el puerto 4321 con Bun. En Kubernetes se sirve mediante NGINX en el puerto 80.

API Gateway: KrakenD reemplaza al BFF tradicional. Toda la lógica de enrutamiento se define en un archivo JSON. Expone 8 endpoints, maneja CORS y propaga el header Authorization a los microservicios que lo requieren.

Microservicios: Tres servicios independientes desarrollados con Hono 4 sobre Bun 1.3.14. Bun incluye test runner, gestor de paquetes y compilador integrados.

- ms-users (:3001) — registro, login, JWT. Hash Argon2 con Bun.password.
- ms-waitlist (:3000) — CRUD de lista de espera con prioridades (1-4) y estados. Valida JWT con @hono/jwt. Publica eventos en RabbitMQ.
- ms-notifications (:3002) — consume RabbitMQ, persiste y envía notificaciones vía WebSocket nativo de Bun.

Persistencia: Cada microservicio tiene su propia base de datos PostgreSQL 16 independiente (Database per Service). Las migraciones se ejecutan al iniciar el contenedor. En Kubernetes se usan StatefulSets con volúmenes persistentes y Jobs de migración independientes.

1.3. Autenticación y Seguridad

La autenticación se implementa mediante JWT con el siguiente flujo:

1. Registro: POST /api/auth/users — valida email único, hashea contraseñas con Argon2 (Bun.password), guarda en users_db.
2. Login: POST /api/auth/users/login — verifica hash, firma JWT con jsonwebtoken (HS256, expiración 2 horas).
3. Payload JWT: { id, email, role_id } — permite identificar al usuario y su rol en cada petición.
4. Validación: ms-waitlist usa @hono/jwt middleware en todas sus rutas protegidas.
5. Ruta /mine: extrae userId del JWT en lugar de recibirlo por query param, evitando acceso no autorizado a datos de otros pacientes.

En cuanto a seguridad, se consideraron varios aspectos: las contraseñas nunca se almacenan en texto plano gracias al hash Argon2; el JWT tiene expiración de 2 horas para limitar el ventana de exposición; el API Gateway (KrakenD) centraliza el control de acceso y maneja CORS para permitir solo orígenes específicos; y cada microservicio tiene su propia base de datos, evitando que un compromiso afecte todo el sistema.

1.4. Escalabilidad y Sostenibilidad

La arquitectura de microservicios fue diseñada pensando en escalabilidad horizontal. Cada servicio puede escalarse de manera independiente según la demanda: si hay muchos registros en lista de espera, se puede duplicar ms-waitlist sin afectar a ms-users. En Kubernetes, esto se gestionaría mediante Horizontal Pod Autoscaler (HPA), aunque en la implementación actual se mantiene una réplica por servicio para el entorno de desarrollo.

Desde la perspectiva de sostenibilidad, la elección de Bun como runtime reduce el consumo de recursos en comparación con Node.js, ya que es significativamente más rápido en arranque y usa menos memoria. El frontend con Astro sigue un modelo de Island Architecture que envía cero JavaScript por defecto, reduciendo la huella de carbono de cada carga de página. Las imágenes Docker usan Alpine como base, minimizando el tamaño de los contenedores.

En cuanto a responsabilidad y privacidad, los datos sensibles de pacientes (razón de consulta, estado de salud) se almacenan en bases de datos aisladas. La ruta `/mine` garantiza que cada paciente solo vea sus propios turnos. El sistema de notificaciones solo envía información al usuario destinatario, y la conexión WebSocket requiere un `userId` válido.

2. Decisiones en el Desarrollo de Componentes Backend y Frontend

2.1. Decisiones de Backend

La elección de Bun como runtime se basó en su rendimiento superior en operaciones de E/S, su soporte nativo para TypeScript (sin paso de compilación adicional), y la integración de herramientas como test runner y gestor de paquetes. Esto simplificó el toolchain del proyecto: un solo runtime para ejecución, testing y gestión de dependencias.

Hono fue seleccionado como framework HTTP por su ligereza (pesa menos de 200KB), su compatibilidad con Bun y su API similar a Express que facilita la curva de aprendizaje. A diferencia de frameworks más pesados como NestJS, Hono no impone una estructura de proyecto rígida, lo que permitió organizar el código en carpetas (routes, controllers, services, repositories) de manera natural.

Para la comunicación asíncrona entre `ms-waitlist` y `ms-notifications`, se eligió RabbitMQ por su fiabilidad en la entrega de mensajes, soporte de colas duraderas (los mensajes persisten aunque el consumidor esté offline), y la posibilidad de reintentos automáticos. Esta decisión permitió desacoplar completamente ambos servicios: `ms-waitlist` publica eventos y no necesita saber si hay consumidores activos.

Una decisión importante fue el uso de PostgreSQL como motor de base de datos para los tres servicios. Aunque cada servicio tiene su propia instancia (Database per Service), el uso del mismo motor simplifica las migraciones y el mantenimiento. El driver ``postgres`` (porsager) con

tagged templates permite escribir consultas SQL tipadas sin overhead de un ORM, manteniendo control directo sobre las consultas.

2.2. Decisiones de Frontend

Astro fue elegido como framework principal por su modelo de Island Architecture: renderiza HTML estático por defecto y solo hidrata los componentes interactivos (Preact) cuando el usuario los necesita. Esto resulta ideal para una aplicación de salud donde la mayoría de las páginas son informativas (inicio, quiénes somos) y solo unas pocas requieren interacción (login, lista de espera).

Preact se utilizó en lugar de React por su tamaño (3KB vs 45KB) y compatibilidad casi total con la API de React. Para una aplicación donde el bundle size importa (usuarios con conexiones lentas en zonas rurales), esta reducción es significativa.

Nanostores fue seleccionado para el manejo de estado global por su simplicidad y tamaño mínimo. Comparado con soluciones como Redux o Zustand, Nanostores ofrece atoms reactivos que se actualizan de manera eficiente, y funciona tanto en el servidor como en el cliente.

2.3. Aciertos y Mejoras Identificadas

Entre los aciertos más relevantes se encuentran: la separación en microservicios que permitió desarrollar cada componente de manera independiente; el uso de KrakenD como API Gateway, que eliminó la necesidad de escribir un BFF custom; y la implementación de notificaciones en tiempo real con WebSocket, que mejoró significativamente la experiencia del usuario.

Sin embargo, se identifican varias mejoras potenciales:

- El WebSocket actual no valida JWT, lo que permite que cualquier userId se conecte sin autenticación. Una mejora sería enviar el token en el handshake y validarlo antes de establecer la conexión.
- La conexión a RabbitMQ en ms-waitlist abre y cierra una conexión por cada publicación. Para alta concurrencia, se debería mantener una conexión persistente con channel pooling.
- El frontend no tiene un componente de ruta protegida reutilizable; la validación de autenticación se hace de manera ad-hoc en cada componente.
- La cobertura de tests del frontend podría mejorar: componentes como WaitlistTable, UsersList y Sidebar no tienen tests unitarios.

3. Aplicación de Patrones de Diseño

Se implementaron varios patrones de diseño a lo largo del proyecto, tanto en el backend como en el frontend, con el objetivo de mantener el código modular, reutilizable y testeable.

3.1. Repository Pattern

Cada microservicio implementa un Repository Pattern que encapsula todas las consultas SQL en un módulo dedicado (carpeta repositories/). Los servicios business logic nunca acceden directamente a la base de datos; siempre pasan por el repositorio. Esto aísla la lógica de persistencia y facilita los tests: se mockea el repositorio y se valida la lógica de negocio de manera independiente.

Efectividad: Este patrón demostró ser muy útil durante el desarrollo. Cuando se cambió el driver de PostgreSQL, solo fue necesario modificar los repositorios sin tocar los servicios. Además, los tests del repositorio se pueden escribir por separado validando las consultas SQL.

Desafío: En ms-notifications, el repositorio inicialmente usaba la convención de nombre singular ("repository/") mientras que ms-users y ms-waitlist usaban plural ("repositories/"). Esta inconsistencia fue un desafío menor que se resolvió unificando la nomenclatura.

3.2. Service Layer Pattern

Cada microservicio tiene una capa de servicios que contiene la lógica de negocio. Los controladores son delgados: solo parsean la request, delegan al servicio y formatean la response. Esto mantiene el código de negocio testable sin depender del framework HTTP.

Efectividad: El patrón permitió escribir tests de negocio que no dependen de HTTP. Por ejemplo, las validaciones de prioridad (1-4) y longitud de razón (mínimo 5 caracteres) en ms-waitlist se testean directamente en el servicio sin simular requests HTTP.

3.3. Centralized Error Handler

Se implementó una clase AppError personalizada con un código de estado HTTP, y un errorHandler global que captura todas las excepciones. En los servicios que usan Hono (ms-users, ms-waitlist), se registra con app.onError(). En ms-notifications (Bun.serve nativo), se adapta retornando un Response directamente. Esto garantiza un formato de respuesta de error consistente: { success: false, message: "..." }.

Efectividad: Este patrón eliminó la duplicación de código de manejo de errores en cada ruta. Además, facilita el debugging: todos los errores se loglean con código de estado y mensaje.

3.4. Observer / Pub-Sub Pattern

La comunicación entre ms-waitlist y ms-notifications implementa un patrón Pub-Sub a través de RabbitMQ. ms-waitlist publica eventos (WAITLIST_STATUS_CHANGED) cuando se agrega un paciente o se cambia el estado de un turno. ms-notifications consume estos eventos, los persiste en la base de datos y los envía al usuario en tiempo real via WebSocket.

Efectividad: Este patrón desacopló completamente ambos servicios. ms-waitlist no necesita saber si hay consumidores activos, y ms-notifications puede procesar mensajes en su propio ritmo. Si ms-notifications se cae, los mensajes persisten en la cola y se procesan cuando vuelva a estar disponible (colas duraderas).

Desafío: La implementación actual abre una conexión nueva a RabbitMQ por cada publicación. Una alternativa más eficiente sería mantener una conexión persistente con un pool de canales, mejorando el rendimiento bajo alta concurrencia.

3.5. Atomic Store Pattern (Frontend)

En el frontend, se utiliza Nanostores con un patrón de atoms atómicos. El estado de autenticación (\$token, \$user) y de notificaciones (\$notifications, \$unreadCount) se maneja mediante atoms reactivos que se actualizan de manera eficiente. Los componentes se suscriben a estos atoms usando useStore() de @nanostores/preact, lo que garantiza re-renderizados mínimos.

Efectividad: Este patrón es ideal para la arquitectura Island de Astro, donde cada componente Preact es independiente. Los atoms permiten compartir estado entre componentes sin prop drilling ni context providers complejos.

4. Estrategia de Branching y Gestión de Versiones

4.1. Estrategia Adoptada

El proyecto utiliza una variante de Git Flow con las siguientes ramas principales:

- `main` — rama de producción, estable. Solo recibe merges desde `develop` vía Pull Requests aprobados.
- `develop` — rama de integración. Aquí se consolidan todas las funcionalidades antes de pasar a producción.
- `feature/*` — ramas de funcionalidad, una por cada tarea o historia de usuario (ej: `feature/36-integracion-frontend`, `feature/42-kubernetes-integration`).
- `chore/*` — tareas de mantenimiento (ej: `chore/13-docker-postgres-setup`, `chore/29-add-dockerfiles`).
- `fix/*` — correcciones de bugs (ej: `fix/27-db-healthchecks`).
- `refactor/*` — reestructuración de código sin cambio funcional (ej: `refactor/22-ms-waitlist`).

4.2. Flujo de Trabajo

El flujo de trabajo siguió este patrón:

1. Se crea una rama `feature` desde `develop` con un nombre descriptivo que incluye el número de issue (ej: `feature/36-integracion-frontend`).
2. Se desarrolla la funcionalidad con commits atómicos usando convencional commits (`feat`;, `fix`;, `test`;, `docs`;, `chore`:).
3. Se abre un Pull Request hacia `develop` para revisión.
4. Tras la aprobación, se hace merge y se elimina la rama `feature`.
5. Periodicamente, `develop` se mergea a `main` con un PR de integración.

4.3. Convención de Commits

Se utilizó Convencional Commits para mantener un historial limpio y automatizable:

Prefijo	Uso	Ejemplo
feat	Nueva funcionalidad	<code>feat(waitlist):</code> agregar ruta GET /mine
fix	Corrección de bug	<code>fix(krakend):</code> pasar header Authorization

test	Tests unitarios	test: cobertura >=60% en todos los componentes
docs	Documentación	docs: anadido kubernetes en report.pdf
chore	Tareas de mantenimiento	chore: eliminar Header.tsx (codigo muerto)
refactor	Reestructuración	refactor: hacer configurables BASE_URL y WS_URL
build	Sistema de build	build(ms-notifications): ejecucion de migrate.ts

4.4. Impacto en el Trabajo en Equipo

Esta estrategia permitió que varios miembros del equipo trabajaran en funcionalidades distintas sin conflictos. Por ejemplo, mientras un miembro desarrollaba ms-notifications (feature/31), otro implementaba la integración del frontend (feature/36), y un tercero configuraba Kubernetes (feature/42). Las ramas feature se mantuvieron aisladas y se integraron de manera controlada.

Una dificultad encontrada fue la necesidad de resolver conflictos de merge cuando múltiples ramas modificaban archivos comunes (como docker-compose.yml). Esto se mitigó definiendo responsabilidades claras por rama y comunicando cambios en archivos compartidos.

Posibles mejoras: implementar CI/CD con GitHub Actions para ejecutar tests automáticamente antes de mergear, y proteger la rama main con reglas de branching que requieran aprobación de al menos un revisor.

5. Integración de Componentes Backend, Frontend y Base de Datos

5.1. Comunicación entre Servicios

La comunicación entre componentes se estructura en tres niveles:

Frontend → API Gateway → Microservicios: El frontend envía todas las peticiones REST a KrakenD (puerto 8083), que las enruta al microservicio correspondiente. KrakenD maneja CORS, timeout de 5 segundos, y propaga el header Authorization para las rutas protegidas.

Frontend → ms-notifications (WebSocket): Para notificaciones en tiempo real, el frontend conecta directamente a ms-notifications sin pasar por KrakenD. Esto se debe a que KrakenD no soporta WebSockets de manera nativa. La conexión se establece con el userId como parámetro de query.

ms-waitlist → RabbitMQ → ms-notifications: La comunicación asíncrona entre servicios backend ocurre a través de RabbitMQ. ms-waitlist publica eventos de tipo WAITLIST_STATUS_CHANGED en la cola "notifications_queue" (durable). ms-notifications consume estos eventos, los persiste en notifications_db y los envía al frontend via WebSocket si el usuario está conectado.

5.2. Manejo de Datos

Cada microservicio tiene su propia base de datos PostgreSQL 16 (Database per Service). Esto garantiza el aislamiento: una caída en una base de datos no afecta a los demás servicios. La referencia entre servicios se hace de manera loose coupling: waitlist_entries.user_id contiene el UUID del usuario de ms-users, pero sin foreign key constraint entre bases de datos.

Las migraciones se ejecutan automáticamente al iniciar cada contenedor (CMD: bun run src/db/migrate.ts). El sistema de migraciones es custom: lee archivos .sql del directorio migrations/, los compara contra una tabla _migrations_history para evitar re-ejecuciones, y ejecuta los pendientes.

5.3. Desafíos de Integración

El principal desafío de integración fue la conexión WebSocket. KrakenD no soporta WebSockets, por lo que el frontend necesita conectarse directamente a ms-notifications. En Kubernetes, esto requiere una regla de Ingress separada para la ruta /ws con anotaciones de timeout extendido (proxy-read-timeout: 3600s).

Otro desafío fue la integración de RabbitMQ. El productor (ms-waitlist) y el consumidor (ms-notifications) deben acordar la estructura del mensaje y el nombre de la cola. Se implementó un esquema de validación en el consumidor que descarta mensajes con campos requeridos faltantes (userId, type, message) y reencola mensajes con errores transitorios.

La integración del frontend con el backend también presentó desafíos. El frontend corre como aplicación cliente (no SSR), por lo que las variables de entorno (PUBLIC_API_URL, PUBLIC_WS_URL) se inyectan en tiempo de build. En Docker, se usan build args; en Kubernetes, se inyectan al construir la imagen. Esto generó inconsistencias menores entre puertos (8080, 8081, 8083) que se resolvieron estandarizando los valores según el entorno.

6. Pruebas Unitarias y Aseguramiento de la Calidad

6.1. Estrategia de Pruebas

Se implementaron pruebas unitarias en los cuatro componentes del sistema, utilizando herramientas nativas de cada ecosistema:

Componente	Framework	Tests	Cobertura Líneas	Cobertura Funciones
ms-users	bun:test	19 tests (3 archivos)	98.43%	96.88%
ms-waitlist	bun:test	19 tests (4 archivos)	94.67%	87.04%
ms-notifications	bun:test	20 tests (4 archivos)	99.35%	100%
Frontend	Vitest + @testing-library/react	36 tests (7 archivos)	97.97%	90.62%

El enfoque de testing siguió la pirámide de tests: pruebas unitarias en la base (mayor cantidad), validando cada capa de manera aislada. No se implementaron tests de integración o E2E, lo que representa un área de mejora.

6.2. Estrategia de Mocking

Cada capa se testea de manera aislada usando diferentes técnicas de mocking:

- Repositorios: se mockea el cliente ``sql`` (tagged template) interceptando las consultas y validando que los parámetros sean correctos.
- Servicios: se mockea el repositorio usando `spyOn()` o `mock.module()` para inyectar datos de prueba.
- Controladores: se mockea el servicio y se simulan requests HTTP usando `app.fetch()` para validar códigos de estado y estructura de respuesta.
- RabbitMQ: se mockea `amqplib` completo usando `mock.module()` para simular conexión, canales y mensajes.
- Frontend (API): se mockea `fetch` global con `vi.stubGlobal()` para simular respuestas del backend.
- Frontend (WebSocket): se crea una clase fake de `WebSocket` que simula eventos `onopen`, `onmessage` y `onclose`.

6.3. Impacto en la Calidad del Código

Las pruebas unitarias tuvieron un impacto directo en la calidad del código. Al tener cobertura superior al 94% en todos los componentes, se detectaron y corrigieron errores antes de la integración. Por ejemplo, en `ms-notifications` se validó que mensajes venenosos (con campos faltantes) se descarten correctamente sin bloquear la cola, y que errores de base de datos generen reintentos automáticos.

En el frontend, los tests del API client validaron que el envelope pattern (`{ success, data }`) se maneje correctamente, y que los headers de autenticación se propaguen en cada petición. Los tests del hook `useNotifications` validaron la reconexión automática con backoff exponencial.

6.4. Dificultades y Estrategias de Optimización

Las principales dificultades encontradas fueron:

Mocking del cliente postgres: El driver ``postgres`` (`porsager`) usa tagged templates para las consultas SQL, lo que dificulta el mocking tradicional. Se resolvió interceptando la función ``sql`` como tagged template, reconstruyendo la query a partir de los string segments y los parámetros.

Testing de WebSocket: El WebSocket nativo de Bun no se puede testear directamente en el entorno de test. Se creó una clase fake que simula el comportamiento del WebSocket, permitiendo validar la lógica de reconexión y procesamiento de mensajes.

Tests del frontend: El entorno happy-dom no soporta todas las APIs del navegador. Se implementó un mock de localStorage en test-setup.ts que simula las operaciones de almacenamiento.

Para futuras implementaciones, se sugiere: agregar tests de integración que validen la comunicación entre servicios end-to-end; implementar E2E tests con Playwright o Cypress para validar flujos completos de usuario; y establecer un pipeline de CI/CD que ejecute los tests automáticamente en cada PR.

7. Conclusión

La arquitectura implementada para RedNorte ofrece una solución desacoplada basada en microservicios que responde a las necesidades de gestión de listas de espera del Servicio Público de Salud. KrakenD como API Gateway elimina la necesidad de un BFF tradicional, los tres microservicios están aislados con bases de datos independientes siguiendo el principio de responsabilidad única, y RabbitMQ permite la comunicación asíncrona garantizando la entrega de notificaciones incluso ante caídas temporales.

Los patrones de diseño aplicados (Repository, Service Layer, Centralized Error Handler, Pub-Sub, Atomic Store) demostraron ser efectivos para mantener el código modular y testeable. La estrategia de branching con Git Flow y convencional commits facilitó la coordinación del equipo y mantuvo un historial limpio. Las pruebas unitarias, con más de 94 tests y cobertura superior al 94% en todos los componentes, contribuyeron significativamente a la estabilidad del código.

El sistema soporta dos modos de despliegue: Docker Compose para desarrollo y presentaciones, y Kubernetes con Kustomize para producción, incluyendo NGINX para el frontend, StatefulSets para bases de datos, ConfigMaps para configuración, e Ingress para enrutamiento con soporte WebSocket. Las áreas de mejora identificadas incluyen: autenticación en WebSocket, tests de integración, CI/CD automatizado, y optimización de la conexión a RabbitMQ.